Multi-Region Architecture

every region on its own, every record one home

Watch the 4-min video

In one line: going multi-region means choosing active-passive or active-active, accepting that cross-region latency makes synchronous writes slow, and avoiding write conflicts, usually by giving every user or tenant one home region, while keeping regions independent of each other.

1 Two questions from the CEO

The app runs well in one region. Then: "what if the whole region goes down?" (lesson 12), and "why are pages slow for customers in Asia?" Going multi-region answers both, but it changes almost everything about how you handle **data**.

2 Two shapes

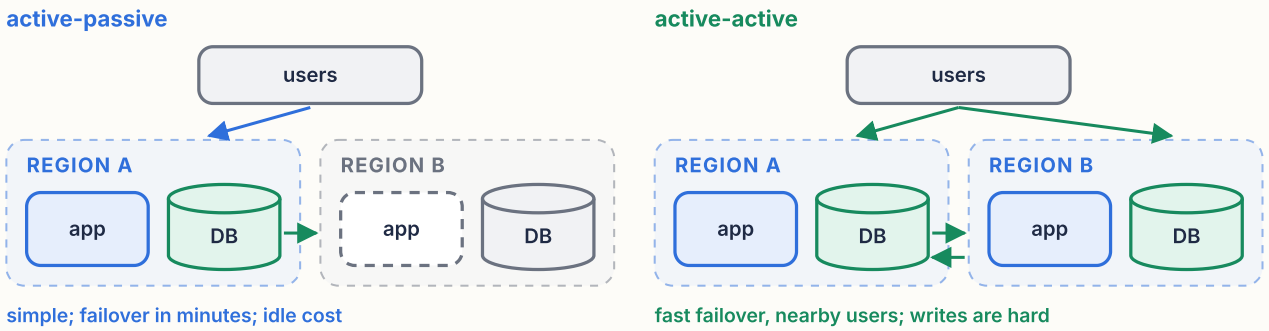


Figure 1. Active-passive keeps a standby region with replicated data; active-active serves traffic from every region at once.

Active-passive: one region serves all traffic, a second stands by with replicated data. Simpler, but failover takes minutes and the passive region is idle cost. **Active-active**: every region serves at once, so users get a nearby region and failover is just routing, but now two regions can write.

3 Why writes are hard

Regions are tens to hundreds of milliseconds apart (roughly 70–100 ms round trip between the US East Coast and Western Europe). **Synchronous** replication adds that to every write. **Asynchronous** replication is fast, but two regions can change the same record at once, and you get conflicts.

Approach	How	Cost
Home region per user/tenant	each record has exactly one writer region; reads may be local	re-homing on failure; cross-tenant features are harder
Synchronous / consensus	commit across regions (e.g. Spanner, CockroachDB)	cross-region latency on every write
Last writer wins	keep the write with the latest timestamp	silently drops a concurrent update
CRDTs / merge	data types that merge concurrent changes (counters, sets)	only fits some data; more complex

4 Home regions, isolation and sovereignty

The most common design avoids conflicts: each user or tenant has a **home region** where all its writes go. Data of different tenants never conflicts. Keep regions **isolated like cells** (lesson 16): no cross-region calls in the request path, so a failure in one can't spread. Home regions also make **data residency** natural, for example keeping EU customers' data in the EU when GDPR or a contract requires it.

WORKED EXAMPLE: A SAAS APP IN THREE REGIONS

The app runs in the US, Europe and Asia. At sign-up each tenant gets a **home region**, stored in a small, highly replicated global directory. A routing layer looks up the tenant and sends every request to its home region, where all reads and writes happen. Each region's data is replicated **asynchronously** to a backup region. If Europe fails, its tenants are re-homed to the backup, accepting the replication lag, a few seconds of data, as loss.

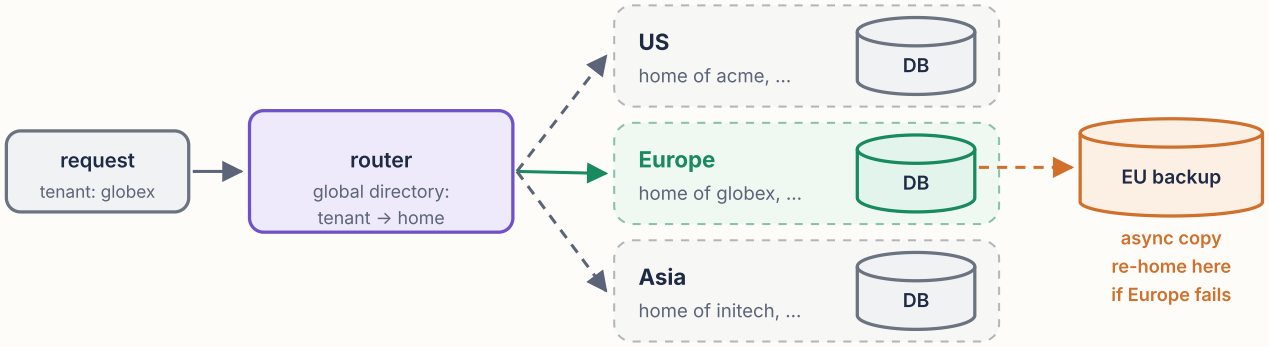


Figure 2. Routing by tenant: every tenant's data has one writer region, so there are no write conflicts to resolve.

THE COSTS

- Duplicate infrastructure, plus cross-region data transfer charges.
- Deploys, monitoring and on-call across regions; config drift between them.
- Global pieces (identity, DNS, the tenant directory) need their own careful design.
- Go multi-region for a real need: availability, latency or compliance, not because it sounds robust.

5 Key terms

Active-passive: one region serves; another stands by

Active-active: all regions serve traffic at once

Home region: the one region that owns a user's or tenant's writes

Write conflict: two regions change the same record concurrently

CRDT: data type whose concurrent updates merge automatically

Data residency: rules on where data may be stored

6 Interview questions

Q1. Active-passive vs active-active? BASIC

Active-passive serves from one region with a standby that takes over in minutes; active-active serves from all regions at once, giving low latency and fast failover but hard writes.

Q2. Why not replicate synchronously across regions? BASIC

Every write would wait for a cross-region round trip, tens to hundreds of milliseconds, and an unreachable region could block writes.

Q3. What changes when you go multi-region? SCENARIO

Cross-region latency makes synchronous writes slow, so you choose active-passive or active-active, design for replication lag and conflicts, isolate regions, and pay more in cost and complexity.

Q4. How do you handle writes in two regions? SCENARIO

Avoid conflicts with a home region per user or tenant (one writer region per record), or resolve them with last-writer-wins or CRDTs.

Q5. Your regions call each other's APIs for some requests. Problem? TRICKY

Yes: it couples their availability and latency, so one region's outage spreads to the other. Each region should serve its requests on its own.

Q6. What's the risk of last-writer-wins? TRICKY

Concurrent updates are silently lost, and clock skew between regions can make the 'last' write the wrong one.

Next: SD-K-016 · Cell-Based Architecture backendroadmap.com